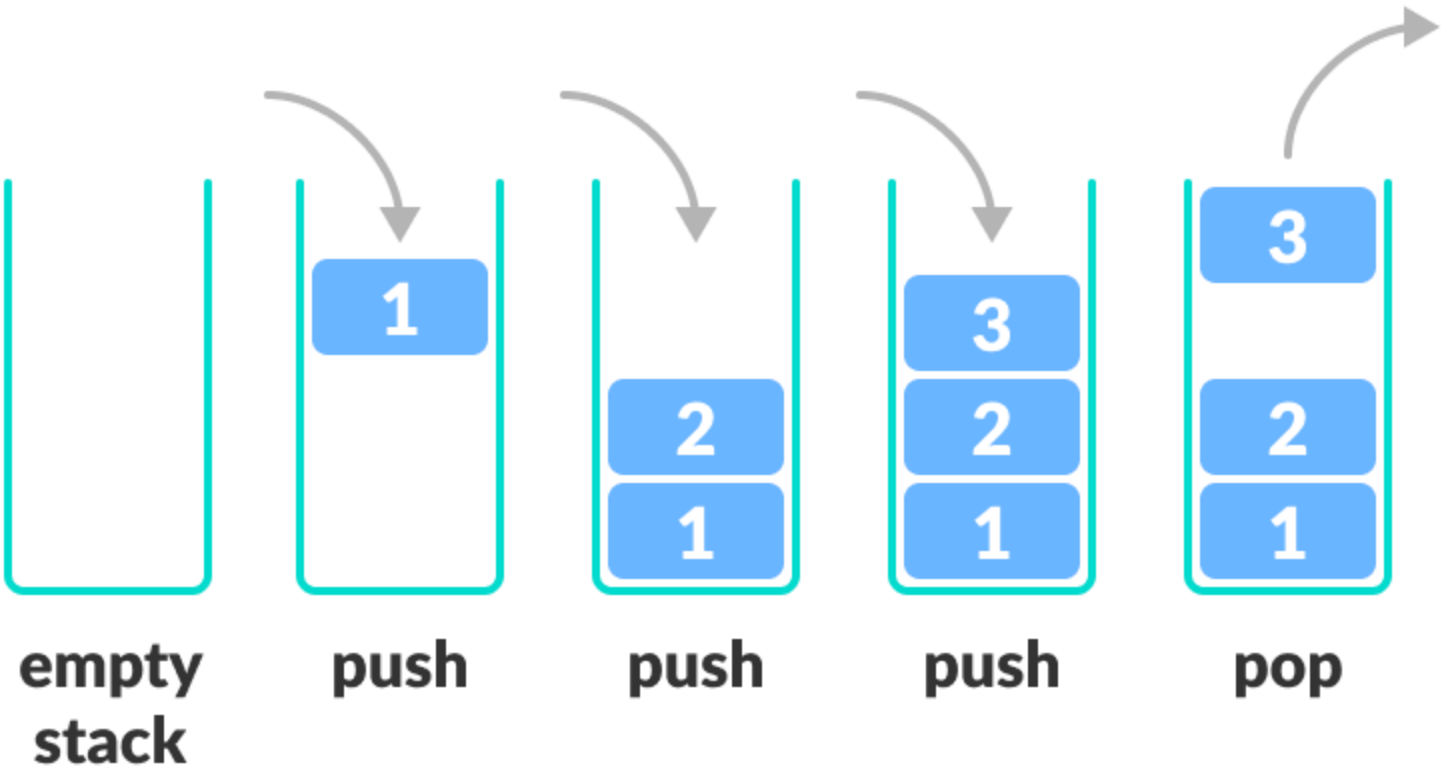
Section 3

Micro CC421

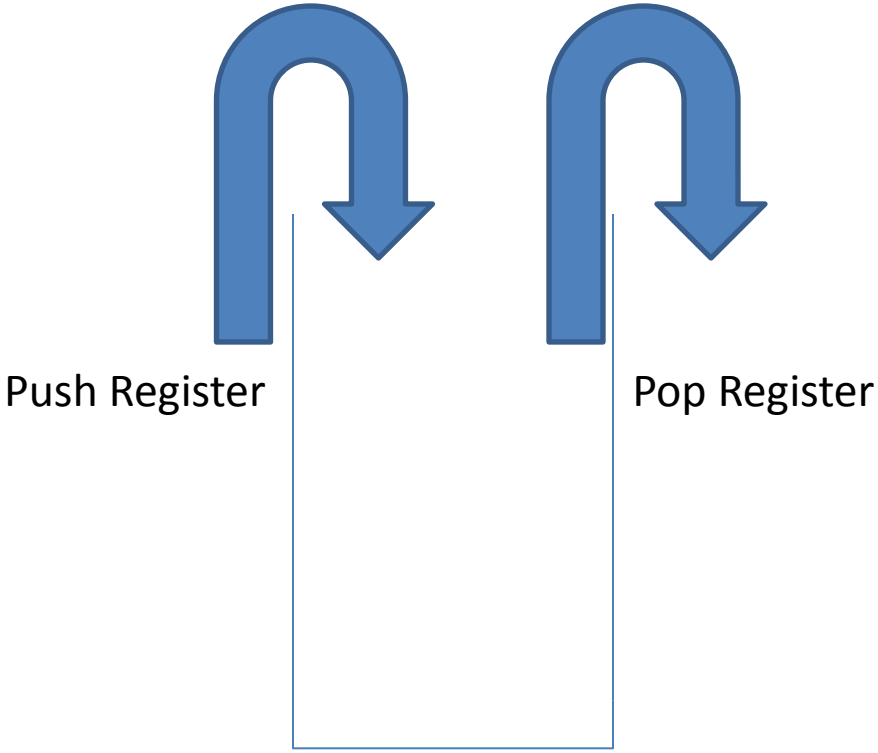
Stack



Stack instructions

Push Register; Must be 16 bits register

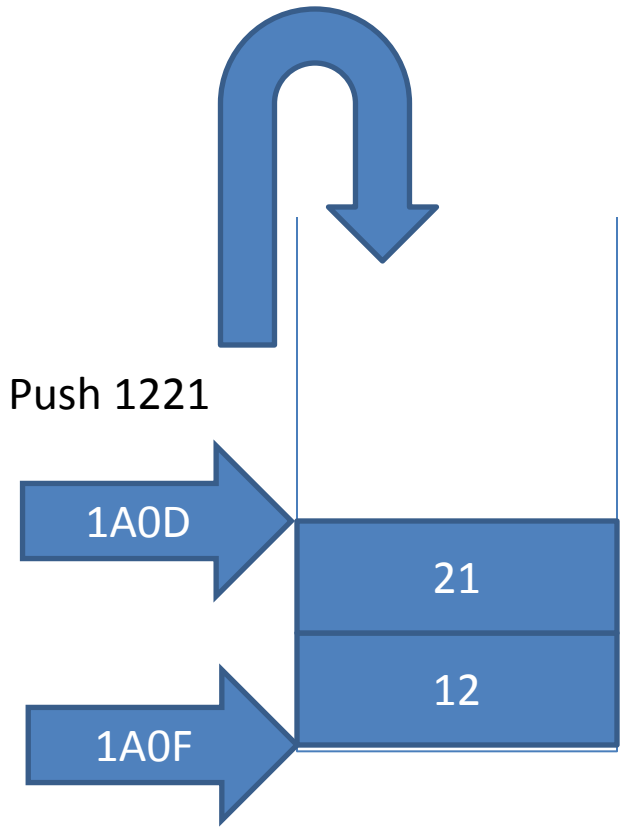
Pop Register; Must be 16 bits register



Example Ax=1221, bx=4321, sp=1A0F

Push AX;

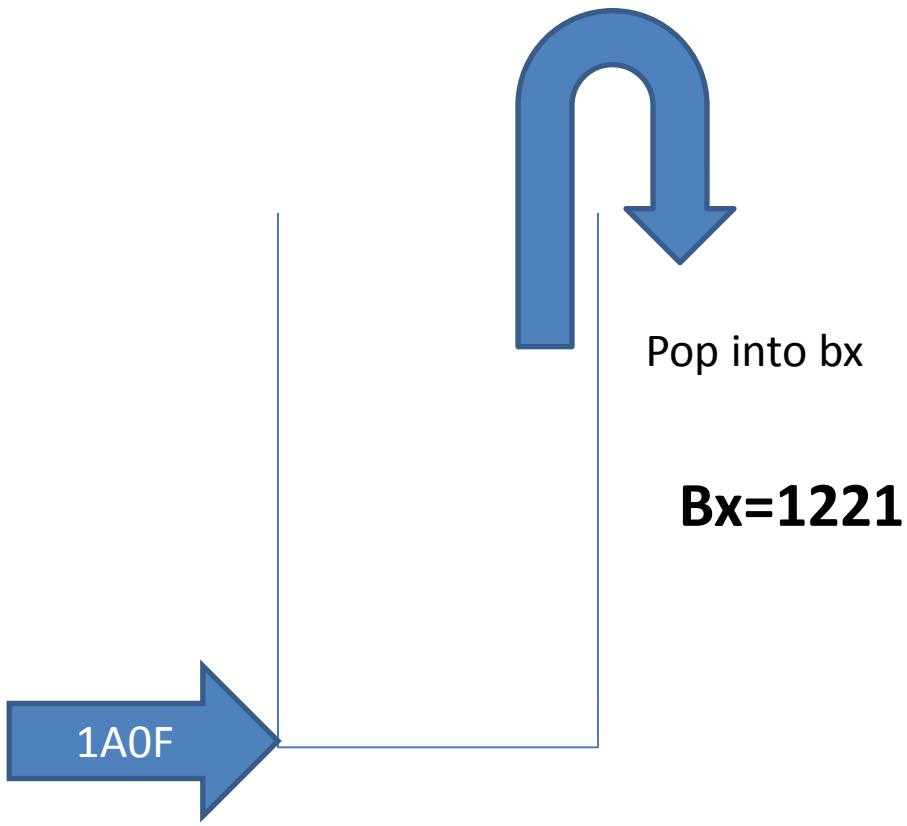
Pop Bx;



Example Ax=1221, bx=~~4321~~, sp=1A0F

Push AX;

Pop Bx;



1. The stack is:

(a) A section of ROM
(b) A section of RAM used for temporary storage
(c) A16-bit register inside the CPU
(d) Some memory inside the CPU

1. The stack is:

(a) A section of ROM
(b) A section of RAM used for temporary storage
(c) A16-bit register inside the CPU
(d) Some memory inside the CPU

1. The stack pointer is:

(a) A section of ROM

(b) A section of RAM used for temporary storage

(c) A16-bit register inside the CPU

(d) Some memory inside the CPU

1. The stack pointer is:

(a) A section of ROM

(b) A section of RAM used for temporary storage

(c) A16-bit register inside the CPU

(d) Some memory inside the CPU

3. When data is pushed onto the stack, the stack pointer is _____, but when data is popped off the stack, the stack pointer is _____.

3. When data is pushed onto the stack, the stack pointer is Decremented, but when data is popped off the stack, the stack pointer is incremented.

4. Choose the correct statement:

(a) The stack segment and code segment start at the same point of read/write memory and grow upward.

(b) The stack segment and code segment start at opposite points of read/write memory and grow toward each other.

(c) There will be no problem if the stack and code segments meet each other.

4. Choose the correct statement:

(a) The stack segment and code segment start at the same point of read/write memory and grow upward.

(b) The stack segment and code segment start at opposite points of read/write memory and grow toward each other.

(c) There will be no problem if the stack and code segments meet each other.

5. What is the main disadvantage of the stack as temporary storage compared to having a large number of registers inside the CPU?

5. What is the main disadvantage of the stack as temporary storage compared to having a large number of registers inside the CPU?

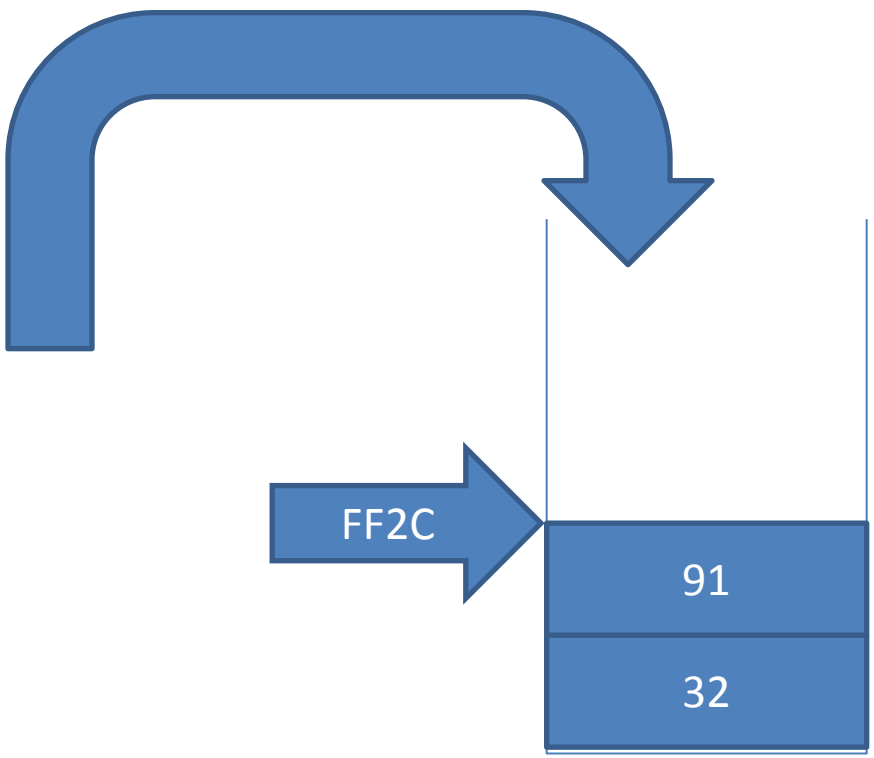
Stack has a much higher access time.

Assume that SP = FF2EH, AX = 3291H, BX = F43CH, and CX = 09. Find the content of the stack and stack pointer after the execution of each of the following instructions.

```
PUSH AX
PUSH BX
PUSH CX
```

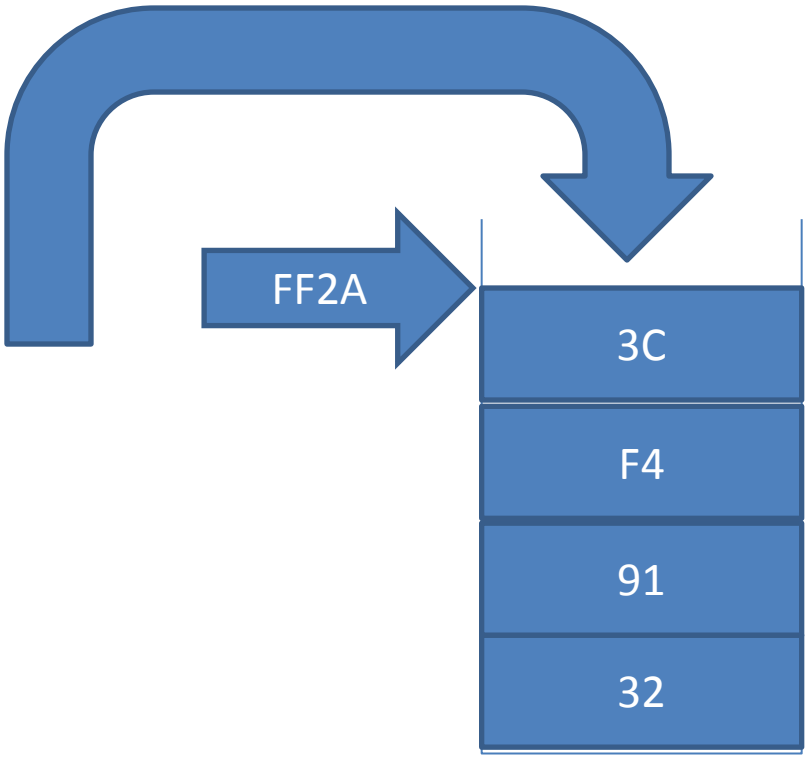
Assume that SP = FF2EH, AX = 3291H, BX = F43CH, and CX = 09. Find the content of the stack and stack pointer after the execution of each of the following instructions.

```
PUSH AX
PUSH BX
PUSH CX
```



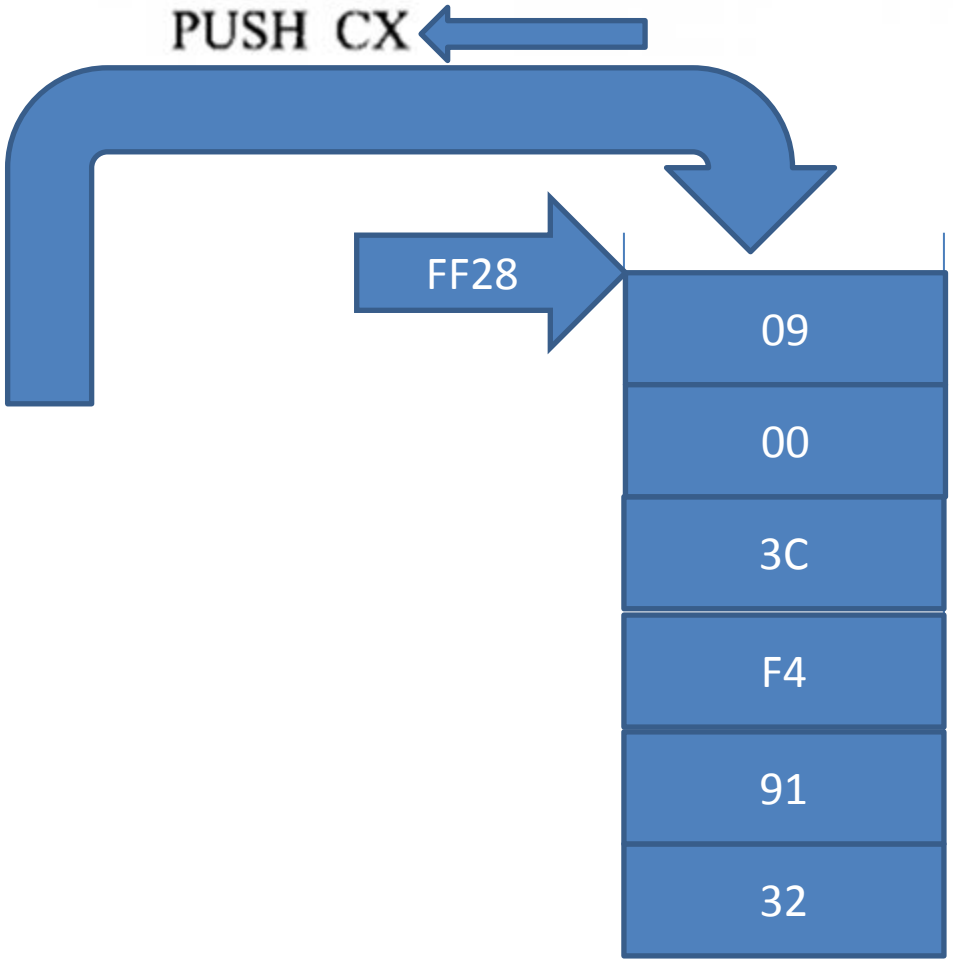
Assume that SP = FF2EH, AX = 3291H, BX = F43CH, and CX = 09. Find the content of the stack and stack pointer after the execution of each of the following instructions.

```
PUSH AX
PUSH BX
PUSH CX
```



Assume that SP = FF2EH, AX = 3291H, BX = F43CH, and CX = 09. Find the content of the stack and stack pointer after the execution of each of the following instructions.

```
PUSH AX
PUSH BX
PUSH CX
```



How to restore the stack without affecting the registers

How to restore the stack without affecting the registers

Pop Cx

Pop Bx

Pop Ax

10. The following registers are used as offsets. Assuming that the default segment is used to get the logical address, give the segment register associated with each offset.

(a) BP (b) DI (c) IP (d) SI (e) SP (f) BX

Segment Register : Offset Register

Table 1-3: Offset Registers for Various Segments

Segment register:	CS	DS	ES	SS
Offset register(s):	IP	SI, DI, BX	SI, DI, BX	SP, BP

10. The following registers are used as offsets. Assuming that the default segment is used to get the logical address, give the segment register associated with each offset.

(a) BP (b) DI (c) IP (d) SI (e) SP (f) BX

a. BP -> SP
b. DI -> DS, ES
c. IP -> CS
d. SI -> DS, ES
e. SP -> SS
f. BX -> DS, ES

11. Show the override segment register and the default segment register used (if there were no override) in each of the following cases.

(a) MOV SS: [BX], AX (b) MOV SS: [DI], BX (c) MOV DX, DS: [BP+6]

11. Show the override segment register and the default segment register used (if there were no override) in each of the following cases.

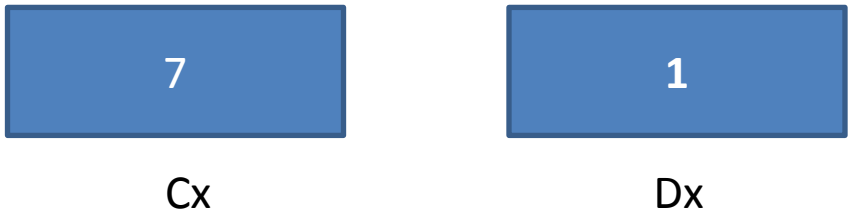
(a) MOV SS: [BX], AX (b) MOV SS: [DI], BX (c) MOV DX, DS: [BP+6]

a. Override (default DS or ES)
b. Override (default DS or ES)
c. Override (default SS)

Add instruction

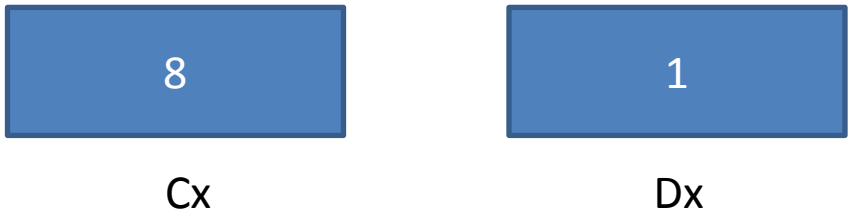
Add Ax, Bx means Ax= Ax + Bx

Example



Example

Add Cx , Dx ; Cx=Cx+Dx



Flags

CF, the Carry Flag . This flag is set whenever there is a carry out, either from d7 after an 8-bit operation, or from d15 after a 16-bit data operation.

PF, the Parity Flag. After certain operations, the parity of the result's low-order byte is checked. If the byte has an even number of 1s, the parity flag is set to 1; otherwise, it is cleared.

AF, Auxiliary Carry Flag. If there is a carry from d3 to d4 of an operation, this bit is set; otherwise, it is cleared (set equal to zero). This flag is used by the instructions that perform BCD (binary coded decimal) arithmetic.

ZF, the Zero Flag. The zero flag is set to 1 if the result of an arithmetic or logical operation is zero; otherwise, it is cleared.

SF, the Sign Flag. Binary representation of signed numbers uses the most significant bit as the sign bit. After arithmetic or logic operations, the status of this sign bit is copied into the SF, thereby indicating the sign of the result.

8 bit Flag example

Example 1-11

Show how the flag register is affected by

```
MOV   AL,9CH      ;AL=9CH
MOV   DH,64H      ;DH=64H
ADD   AL,DH       ;now AL=0
```

Solution:

```
                  CF      AF
                 1 1 1 1 1   1
      9C          1001   1100
+     64          0110   0100
      00          0000   0000
                  SF
```

CF=1 since there is a carry beyond d7
PF=1 since there is an even number of 1s in the result
AF=1 since there is a carry from d3 to d4
ZF=1 since the result is zero
SF=0 since d7 of the result is zero

16 bit flags example

Example 1-12

Show how the flag register is affected by

```
MOV   AX,34F5H     ;AX= 34F5H
ADD   AX,95EBH     ;now AX= CAE0H
```

AF

Solution:

No Carry

```
                    11     1   11    1111    111
      34F5          0011   0100  1111  0101
+     95EB          1001   0101  1110  1011
      CAE0          1100   1010  1110  0000
```

SF

PF

CF = 0 since there is no carry beyond d15

PF = 0 since there is an odd number of 1s in the lower byte

AF = 1 since there is a carry from d3 to d4

ZF = 0 since the result is not zero

SF = 1 since d15 of the result is one

Thanks